

Implementation of SSD: Single Shot MultiBox Detector

Dheeraj Kumar Maradana

Our team has also written a blog on this paper

<https://medium.com/@dheerajkumarmaradana/ssd-single-shot-multibox-detector-d7d570bbbe6f>

1 Introduction

I have implemented SSD300 from scratch using only native PyTorch modules, as well as directly using the SSD module from TorchVision. The official implementation used its own C/C++ modules for the backend and Python for the model. I used the same dataset as the official implementation; hence, I didn't rerun it.

2 Dataset

Dataset Used: Pascal VOC 2007

- Train split: trainval: 5011 images
- Test split: test: 4952 images
- Number of classes: 21 (including background)

3 Architecture Details

3.1 Backbone Network

Used exactly the same network architecture presented in the paper

3.2 Feature Maps

SSD uses multi-scale feature maps:

Layer	Size	Channels
conv4_3	38×38	512
fc7	19×19	1024
conv8_2	10×10	512
conv9_2	5×5	256
conv10_2	3×3	256
conv11_2	1×1	256

Table 1: SSD Feature Maps

3.3 Default Boxes (Anchors)

- Total anchors: 8732
- Aspect ratios: $\{2\}, \{2, 3\}$ depending on layer
- Scales: from 0.07 to 1.05 linearly (this is according to the paper, but the official implementation uses slightly different ones)

3.4 Prediction Heads

Each feature map predicts:

- Classification scores
- Bounding box offsets

Implemented via:

- Conv2D layers with kernel size 3
- Output reshaped to $(N, H \cdot W \cdot A, K)$

4 Loss Function

The SSD loss consists of:

$$L = \frac{1}{N}(L_{loc} + L_{conf}) \quad (1)$$

- Localization loss: Smooth L1
- Confidence loss: Softmax Cross Entropy
- Hard negative mining ratio: 3:1

5 Training Setup

5.1 From-Scratch Implementation and also Torchvision Implementation

- Optimizer: AdamW
- Learning rate: 2e-4 up to 150, 2e-5 from 150 to 180, 2e-6 from 180 to 200
- Batch size: 256
- Epochs: 200
- Weight decay: 1e-4
- Hardware: 1 Nvidia A6000 GPU

5.2 Official Implementation

- Optimizer: SGD
- Learning rate: 1e-3
- Batch size: 32(across 4 GPU's)
- Epochs: 120,000
- Hardware: 8 Titan X GPU's

6 Evaluation Metric

Both use Mean Average Precision (mAP) using VOC 2007 11-point interpolation

7 Results

7.1 Quantitative Results

Method	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
Original	68.0	73.4	77.5	64.9	59.0	38.9	75.2	80.8	78.5	46.0	67.8	69.2	76.6	82.1	77.0	72.5	41.2	64.2	69.1	78.0	68.5
From Scratch	43.27	56.52	55.13	23.59	25.47	10.24	53.46	59.88	47.17	20.94	36.18	49.67	46.22	62.85	61.16	53.21	18.40	37.11	40.20	65.01	42.95
Torchvision	44.30	55.27	58.32	25.81	27.39	10.37	53.65	60.21	49.37	21.05	42.11	44.43	46.07	64.79	63.84	53.94	19.91	37.79	40.17	64.76	46.85

Table 2: Comparison of detection performance (mAP and per-class AP).

From-scratch implementation and Torchvision Implementation produce almost the same results, which proves my implementation is accurate. The difference between the original and my implementation is due to the number of epochs: the original was trained for 120,000 epochs, whereas my implementation was trained for only 200 epochs!